

Anatomia de uma Função Lambda

Handlers, Runtime e Estrutura

AWS Certified Developer Associate

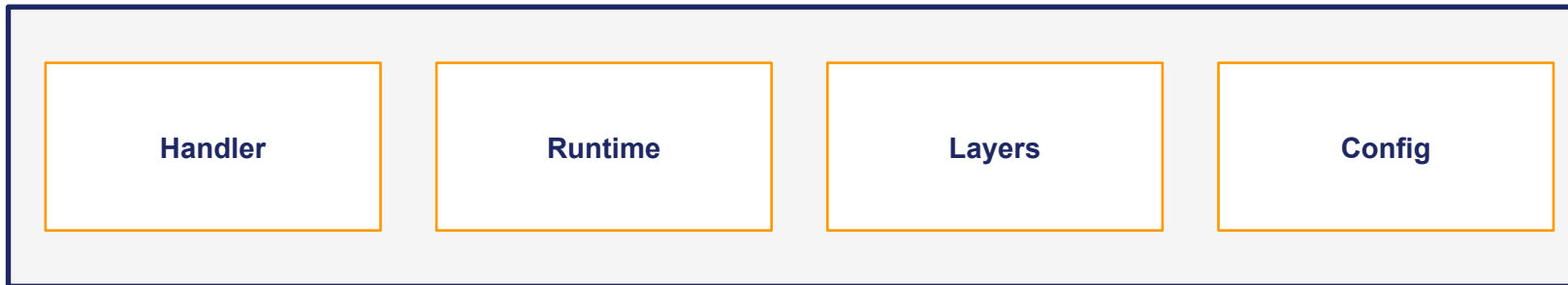
Instrutor: Marlon Anesi

Componentes de uma Função Lambda

Uma função Lambda é composta por:

- Handler: Ponto de entrada do código (função que processa o evento)
- Runtime: Ambiente de execução (Python, Node.js, Java, etc.)
- Deployment Package: Código + dependências (ZIP ou Container)
- Configuração: Memória, timeout, variáveis de ambiente
- Execution Role: Permissões IAM da função

Função Lambda



O Handler - Ponto de Entrada

O handler é a função que o Lambda chama quando invocado.

Formato: `arquivo.funcao` (ex: `index.handler`, `main.lambda_handler`)

Python (`lambda_function.py`)

```
def lambda_handler(event, context):
    # event: dados de entrada
    # context: info do runtime

    name = event.get('name', 'World')
    return {
        'statusCode': 200,
        'body': f'Hello {name}!'
    }
```

Node.js (`index.js`)

```
exports.handler = async (event) => {
    // event: dados de entrada

    const name = event.name || 'World';
    return {
        statusCode: 200,
        body: `Hello ${name}!`
    };
};
```

Handler padrão: `lambda_function.lambda_handler` (Python) ou `index.handler` (Node.js)

Event e Context Objects

Event Object

Dados de entrada da invocação

Conteúdo varia por fonte:

- API Gateway: headers, body, path
- S3: bucket, key, evento
- SQS: Records com mensagens
- Formato JSON

Context Object

Info do ambiente de execução

Propriedades úteis:

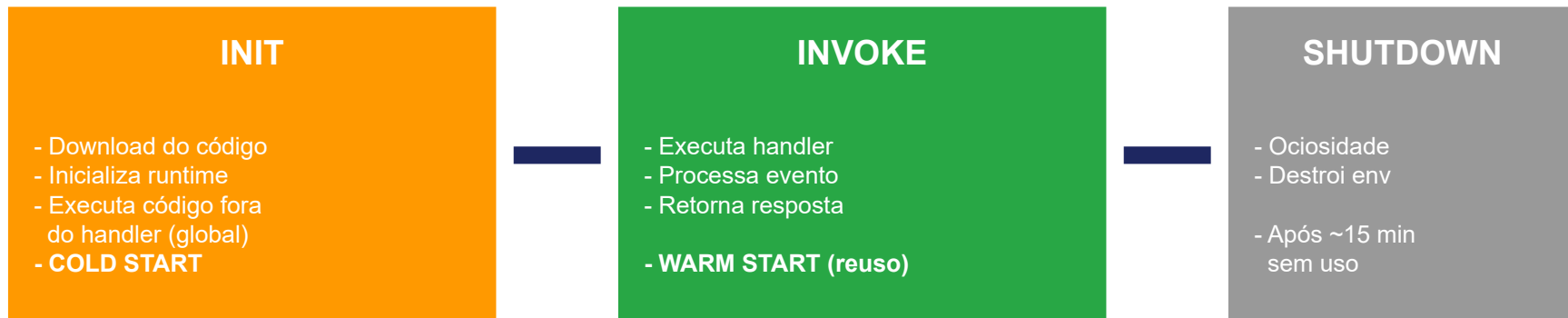
- function_name
- memory_limit_in_mb
- aws_request_id
- get_remaining_time_in_millis()

Dica para prova: Use `context.get_remaining_time_in_millis()` para evitar timeout!

Runtimes Disponíveis:

Python | Node.js | Java | .NET | Go | Ruby | Custom Runtime (bootstrap)

Ciclo de Vida da Execução



Cold Start vs Warm Start:

- Cold Start: Primeira invocação - inclui fase INIT (mais lento)
- Warm Start: Invocações subsequentes - reusa ambiente (mais rápido)
- Dica: Mantenha conexões DB fora do handler para reutilização!

Código fora do handler (escopo global) executa apenas no INIT!



Pontos-chave para a Prova

1. Handler

- Formato: arquivo.funcao
- Recebe event (dados) e context (runtime info)

2. Ciclo de Vida

- INIT (cold start) -> INVOKE -> SHUTDOWN
- Código global executa apenas no INIT
- Conexões DB no escopo global = reutilizadas!

3. Context Object

- `get_remaining_time_in_millis()` para timeout
- `aws_request_id` para tracing

LEMBRE-SE: Inicialização fora do handler = executa apenas 1x no cold start!